interface. The line between this and behavioral tests can be rather blurry, as it is often hard to prove a feature works without interacting with the UI. At this level, you typically need custom tools that act like a user, interacting with the UI and clicking things.

The further up the test pyramid you travel, the slower and more expensive your tests become. Consider a web page where the user inputs their address and sees their cost of shipping calculated. To properly test this, you need all the variations on types of address (houses, apartments, rural routes, PO boxes, etc.) and all the variations on shipping zones. This could result in hundreds of test executions. If you test at the UI level, every single test execution requires the information to be entered in the browser; the data sent to the server; the server to do work; a look-up to happen in the database; the server to formulate a result; and, finally, the result to be displayed in the browser. This round trip is slow and requires you to run both a server and a browser. The vast majority of the logic in this example is in the backend, where the shipping cost gets calculated. By moving the testing down to the unit level, you skip most of the steps, making the test run significantly faster.

When you design tests, you want to appropriately select the level of testing. The vast majority of your tests should be at the unit test level. UI tests should focus on UI features, rather than using the UI as the testing interface. As much as possible, push your tests as far down the pyramid as they can go.

Speedy tests are important. If you test once a week, and something fails, it is hard to remember what you did to the code a week ago that caused the problem. If you test on every check-in of your code, possibly several times a day, you'll be more likely to remember what you were doing that caused the failure. Testing more frequently is good; slow tests make you not want to do this. One reason for using a hierarchy of test categories is to set different test frequencies at different levels. Unit tests are small and fast and done at every check-in. Integration tests might get done once or twice a day, and behavioral and UI tests could be part of a nightly process.

Most of this chapter focuses on unit tests. Behavioral and UI tests require additional tools that are beyond the scope of this book. The line between unit tests and integration test is fuzzy in the Django world, as bigger views likely call several functions. A common unit-test point for Django code is the view itself.

Test-driven development and behavior-driven development

Two testing philosophies have become quite popular in the software world: *test-driven development* (TDD) and *behavior-driven development* (BDD). These ideas are based more around when you write your tests than how. TDD and BDD state that you should write your tests before you write your code. The idea is to decide on a function's interface, write the tests for it, and then write the function. When all a function's tests pass, you are done writing the function. BDD is similar to TDD, with the focus being at the behavioral level, writing requirements-level tests rather than function-level tests.

I am a huge proponent of automated testing and think it is a really important part of modern development. Software teams should set standards for their testing and not